

Assembleia 02 — LLM Gratuito e Banco

Data: 31/08/2026 **Método:** docs/METODO-PLANEJAMENTO.md **Gatilho:** direcionamento direto do Tiago para AI Master Engineer (usar modelos gratuitos + montar acervo de domínio público) e Backend Engineer (reavaliar Supabase vs. alternativa gratuita) **Status:** aguardando aprovação explícita do Tiago — nada implementado

Nota de processo: as três pesquisas iniciais foram delegadas a subagentes em paralelo; o Tiago pediu status três vezes seguidas, elas seguiam running, e a pedido dele foram canceladas e a pesquisa foi refeita por mim mesmo, em série, com WebSearch/WebFetch reais. Todo número e toda afirmação de licença abaixo tem fonte citada — nada saiu de memória.

FRENTE IA — modelos gratuitos + acervo de conteúdo

Consulta: AI Master Engineer

Comparação real de camadas gratuitas de LLM:

Provedor	Limites gratuitos	Contexto	Treina com seus dados?	Fonte
Google Gemini API	Varia por modelo: Flash-Lite 15 RPM/1.000 RPD, Flash 10 RPM/250 RPD, Pro 5 RPM/100 RPD; 250k TPM compartilhado	até 1M tokens	SIM — texto explícito da Google: "usa o conteúdo enviado e as respostas geradas para fornecer, melhorar e desenvolver produtos e serviços Google", com revisão humana. Camada paga NÃO treina.	ai.google.dev/gemini-api/terms (citação direta)

Groq	Genuinamente grátis, sem cartão; limite por modelo (checar painel)	varia por modelo	NÃO — política vale pra conta inteira (grátis e paga igual): não usa inputs/outputs pra treinar/ajustar modelo sem permissão explícita. Retenção de 30 dias só por segurança.	Groq policies/legal; análise de retenção
OpenRouter (modelos :free)	~20 RPM; 50/dia até acumular US\$10 gasto histórico, depois 1.000/dia	varia por modelo roteado	DEPENDE — o OpenRouter em si não guarda prompt/resposta por padrão, mas ele é um *roteador*: a política de quem realmente processa a requisição (o provedor por trás do modelo :free) se aplica ao conteúdo. Vários provedores de modelo gratuito exigem habilitar treino/log pra liberar o modelo — precisa verificar modelo a modelo, não dá pra confiar em bloco.	OpenRouter FAQ; análise 2026
Mistral, Cohere, Hugging Face Inference API	não pesquisado ainda	—	—	pendente

Recomendação do AI Master Engineer: Groq como opção primária. É a única das três verificadas com política de dados limpa (não treina, mesma regra grátis/paga) — resolve de raiz a preocupação que você levantou sobre texto real de jogador passando pela API. O Gemini free tier é explicitamente inadequado pra esse uso: usar ele obrigaria a página do jogo a avisar "seu texto pode ser lido por revisor humano e usado pra treinar modelos do Google", o que é um custo de confiança real pra um jogo onde a pessoa

escreve texto criativo/pessoal. OpenRouter : free fica como opção secundária, só depois de confirmar a política do provedor específico por trás do modelo escolhido — não antes.

Risco a registrar: Groq serve modelos abertos (Llama, Mixtral, DeepSeek etc.), não os modelos Claude — qualidade de narração em português e aderência ao contrato JSON estrito do mestre (`master.js`) precisa ser validada na prática antes de comprometer; ainda não testei isso, é o próximo passo técnico, não uma garantia.

Consulta: Narrative Writer + Code QA Engineer (acervo de domínio público)

Licenças confirmadas, com fonte:

- **SRD 5.1 (D&D 5ª edição)** está de fato sob **Creative Commons CC-BY 4.0** desde 27/jan/2023 — confirmado pelo documento legal oficial da Wizards of the Coast. [PDF oficial](#) · [cobertura](#)
- **Ressalva bloqueante dentro do próprio SRD:** nomes como "Beholder" e "Mind Flayer" aparecem no texto e por isso o *nome* está tecnicamente sob CC-BY, mas o **stat block, a mecânica completa e a descrição detalhada desses monstros específicos continuam de fora**, como propriedade intelectual protegida da WotC — usar o nome não autoriza usar a ficha/mecânica. [Fonte](#) — isso confirma e reforça o anti-padrão que já está no CLAUDE.md original (§14), não muda nada lá, só documenta o porquê.
- **Project Gutenberg:** textos são domínio público nos EUA; **não precisa de permissão pra redistribuir**, com ou sem a marca "Project Gutenberg", **desde que não seja uso comercial cobrado** — se for cobrar pelo conteúdo, precisa remover cabeçalho/rodapé/capa introdutória do Gutenberg. Como o MWRPG não vende o conteúdo em si, a via não-comercial se aplica. [Fonte](#)
- **Domínio Público (Brasil):** prazo de proteção autoral é **70 anos contados de 1º de janeiro do ano seguinte ao falecimento do autor** (Lei 9.610/98) — depois disso a obra é de domínio público automaticamente, mas **direitos morais permanecem** (precisa manter atribuição de autoria, não pode desfigurar a obra). Portal oficial: dominiopublico.gov.br. [Fonte](#)

Nuance importante que registro para o acervo: uma *tradução* ou *coletânea* moderna de um conto folclórico pode ter copyright próprio do tradutor/editor mesmo que a história original seja domínio público — verificar edição por edição, não assumir que "é uma fábula antiga, então tudo bem" cobre a tradução específica usada.

O que ainda falta (não fiz o catálogo em si, só as regras de licença): montar a lista concreta de fábulas/contos/bestiários específicos com URL/número do Gutenberg de cada um, com proveniência registrada por peça — isso é trabalho do Narrative Writer como próxima tarefa, não uma pesquisa de licença.

Proposta de arquitetura do acervo (AI Master Engineer + Narrative Writer)

Em vez do mestre gerar cada NPC/monstro/situação do zero a cada turno, o acervo curado (fábulas, mitologia, bestiários — todos com proveniência registrada) vira **material de composição**: o prompt do mestre recebe trechos relevantes do acervo (recuperação simples por palavra-chave/tag no MVP; RAG de verdade com embeddings fica pra depois, era já o item v0.4 do roadmap original) e narra *a partir* deles em vez de inventar tudo. Isso ataca os dois problemas ao mesmo tempo: menos tokens gerados do zero (custo) e mais qualidade/consistência de mundo (a ideia do Tiago é boa e sustenta isso).

FRENTE BANCO — alternativas gratuitas ao Supabase

Consulta: Backend Engineer + Realtime Multiplayer Engineer + Infra Engineer

Comparação real:

Provedor	Armazenamento grátis	Pausa por inatividade	Tempo real nativo?	Fonte
Supabase (atual)	500MB DB, 1GB storage, 5GB egress	Projeto inteiro pausa após 7 dias sem NENHUMA atividade — não é por sessão, é por semana inteira zerada. Dados intactos, restaura pelo painel.	Sim — Realtime nativo (Postgres Changes + Presence + Broadcast)	detalhamento 2026
Neon (Postgres serverless)	0,5GB storage, 100 horas de computação/mês	Escala a zero após 5 min de inatividade, mas cold start é 300-800ms (bem diferente do Render — não é 30-60s)	Não — precisaria parear com algo à parte	revisão 2026
Turso (SQLite/libSQL na borda)	5GB storage, 500M linhas lidas/mês, 10M escritas/mês	Não pausa da mesma forma (billing por linha lida, não por tempo)	Não — precisaria parear com algo à parte	pricing oficial
Firebase Firestore (Spark)	1GiB storage, 50k leituras/20k escritas/20k exclusões por dia (teto duro, para até o dia seguinte)	Não pausa — teto é diário, não por inatividade	Sim, nativo (onSnapshot) — é o recurso central do Firestore	detalhamento 2026
Abyl (se parear com Neon/Turso)	—	—	200 conexões simultâneas, 6M mensagens/mês grátis	ably.com/docs/platform/pricing/limits

Pusher (se parear com Neon/Turso)	—	—	100 conexões simultâneas, 200k mensagens/dia grátis	Pusher pricing 2026
Appwrite, Convex, PlanetScale, Railway, PocketBase	não pesquisado ainda	—	—	pendente

Correção importante em relação à Assembleia 01: eu tinha caracterizado a preocupação com "hibernação" no mesmo balaio do problema do Render free (que dorme em **15 minutos** de ociosidade — fatal em pleno turno de jogo). O número real do Supabase é **7 dias inteiros sem nenhuma chamada**, não 15 minutos. Uma sala com jogadores ativos nunca chega perto disso — o cenário que mataria a sessão seria o projeto inteiro ficar uma semana sem ninguém jogar, o que se resolve com um simples ping de keep-alive periódico, não é um risco estrutural pra sessão ao vivo. Isso muda a resposta.

Recomendação honesta — ficar no Supabase. As alternativas ou (a) não têm tempo real nativo e exigem somar uma segunda conta/serviço (Ably/Pusher) — mais peça móvel, mais um free tier pra gerenciar, sem ganho real de custo — ou (b) exigem abandonar Postgres/SQL inteiramente por um modelo de documentos (Firebase), o que refaz Auth e o schema do zero sem necessidade concreta. O medo que motivou a pergunta ("sair do Supabase por causa de hibernação") não se sustenta depois de olhar o número real: 7 dias parado, não 15 minutos. Ficar onde está é a resposta honesta aqui, não a mais preguiçosa.

Se algo mudar essa recomendação: se o volume de teste crescer a ponto de estourar 500MB (pouco provável na fase de usuários de teste — é majoritariamente texto), ou se surgir necessidade de mais de 2 projetos ativos simultâneos (limite do free tier), visitar então — não antes.

Resumo para decisão do Tiago

- LLM:** Groq como padrão (grátis, não treina com seus dados), testar qualidade de narração em português antes de comprometer 100%; evitar Gemini free tier pra texto de usuário real por causa da política de treino confirmada; OpenRouter : free só com checagem caso a caso.
- Acervo:** licenças mapeadas e seguras (SRD 5.1 CC-BY 4.0 com a ressalva de nomes-sem-mecânica, Gutenberg, Domínio Público BR) — falta montar o catálogo concreto peça por peça, próximo passo natural do Narrative Writer.
- Banco:** manter Supabase — a razão que motivou reavaliar (hibernação) não corresponde ao comportamento real do free tier.

Nada implementado. Aguardando sua aprovação ou ajuste.